

LEARNSLICE GUIDE

AI for product owners

83% of agile practitioners use AI tools. 15% have had any training in using it in an agile context.

Alesia Kunz, CEO of LearnSlice. 17+ years in software engineering as a product manager and product owner. Built on experience from real teams and projects, and on published research.

Inside: why Copilot cannot read your backlog, the five things that are actually yours, and the prompts worth saving.



Contents

Nearly a third do not know where to start

Part 0: six things nobody told you

1. Your best prompt might have been luck
 2. Your Copilot probably cannot see your work
 3. The four-part prompt Microsoft wrote and nobody reads
 4. The one key most people never find
 5. Where your prompts actually go
 6. It will lie to you in your own house style
-

Part 1: four habits that separate method from luck

- Rule 1. Give it your own material
 - Rule 2. Know what must never go in
 - Rule 3. Check it before it leaves you
 - Rule 4. Keep what works
-

Part 2: the five things that are actually yours

- Chapter 1. Why your items come back from refinement
 - Chapter 2. Refinement is not a group reading session
 - Chapter 3. Ordering is the part nobody can do for you
 - Chapter 4. Planning starts before the meeting does
 - Chapter 5. What got done, and what actually changed
-

Part 3: the prompt pack

Part 4: staying current on thirty minutes a week

- Thirty minutes a week
 - Where to look, and how often
 - The weekly
-

Sources

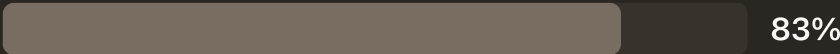
What we do

That is the whole problem. Access is not the constraint. 67% of organisations already provide the tools, and 55% of practitioners still spend a tenth or less of their working time with them. The most cited obstacle is uncertainty about how to integrate it, at 54%, followed by a lack of training material at 36% and not knowing where to start at 31%.

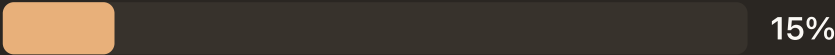
Everyone has it. Almost nobody was taught

Agile practitioners, 2026

Use AI tools



Had any training in using it



67% of their organisations already provide the tools.

Those figures come from the AI4Agile Practitioners Report 2026, 289 practitioners across more than 20 countries. It is a small, self-selected sample that does not break results out by role, so read it as a direction of travel rather than a measurement of product owners specifically. Everything about the role itself comes from the Scrum Guide, and everything about Copilot from Microsoft's own documentation, checked September 2026.

What actually stops people

Not the tool, and not the budget

Not sure how it fits the work 54%

No training material 36%

Ethical and data concerns 35%

Do not know where to start 31%

1% think AI is irrelevant to their work.

Nearly a third do not know where to start

Not knowing where to start is named by 31% of practitioners, so you are in company.

What the tool is, in two sentences: a system that predicts likely text from the text you give it, trained on an enormous amount of writing. That is why it is fluent, why it is useful on anything language-shaped, and why it will state something false with complete confidence.

Start here, once, this week. Take a backlog item you wrote yourself and know well. Ask it to list the questions a developer would ask before picking the item up. You will know immediately whether the answer is any good, which is why you start on work you know.

Part 0: six things nobody told you

Six short sections before the guide proper. Each one takes a minute to read and heads off a specific frustration that otherwise looks like your own fault.

1. Your best prompt might have been luck

Microsoft says it plainly in its own guidance: using the same prompt multiple times can result in different responses. This is how the technology works, not a fault.

The practical consequence is the one people miss. **One good answer does not prove the prompt was good.** It might have been luck. A prompt is only worth keeping once it has produced a usable answer more than once, on different inputs. That is why Rule 4 exists.

2. Your Copilot probably cannot see your work

There are three tiers and they behave completely differently.

What each Copilot can reach

Same name, three different products

	WEB	IN APPS	YOUR FILES
Copilot Chat (Basic)			
Microsoft 365 Copilot (Basic)			
Microsoft 365 Copilot (Premium), paid add-on			

An empty cell means it sees your work only if you paste or upload it.

The same thing in detail, including what each tier can reach and how:

TIER	CAN IT SEE YOUR FILES AND MAIL
Copilot Chat (Basic)	No. Web data only. It sees your work only if you paste it, upload it, or have it open in Teams or Outlook
Microsoft 365 Copilot (Basic)	Not in chat. But Copilot works inside Word, Excel, PowerPoint and OneNote
Microsoft 365 Copilot (Premium), the paid add-on	Yes, automatically, through Microsoft Graph, and only for files you already have permission to open

If you have asked Copilot "what did we agree about this epic" and got nothing useful, you are almost certainly not on the premium tier. That is a licensing fact, not a skill problem.

The model picker. Copilot offers Auto, Quick response and Think deeper. Auto chooses for you. Switch to Think deeper when the task is a judgment rather than a lookup, and expect it to take longer on purpose.

Grounding, which is the word for all of this. Two sources feed any answer: the public web, and your organisation's own content. Only the premium tier reaches the second one by itself. Every other tier sees only what you hand it.

Your backlog probably is not in Microsoft 365. If it lives in Jira or Azure DevOps, no tier of Copilot reads it automatically. Microsoft does publish connectors for Jira and Confluence, but an administrator has to deploy

them and they enforce the source system's own permissions, so a connected source is not the same as a readable project.

Field note. Some Copilot licences offer the Jira integration, and you can check yours under Copilot Chat, then Settings, then Sources. The catch is that even where Jira or Confluence shows up there as an available source, Copilot often still cannot pull data from your particular project. In practice that means product owners and business analysts copy the ticket in by hand to refine a story or its acceptance criteria. Before you paste anything into Copilot or any other AI tool, check that your organisation allows it from a data protection point of view. That check comes before the convenience.

3. The four-part prompt Microsoft wrote and nobody reads

This is Microsoft's own framework, and it is a good one.

Goal. What you want. **Context.** The situation, who it is for, what has already been decided. **Expectations.** Format, length, tone, what not to do. **Source.** Where it should get the material.

A thin prompt: *Split this story.*

The same prompt with four parts: *Split the story below into smaller items. Context: a B2B web product, the team delivers in two-week sprints, and this story is too large to finish in one. Expectations: each item independently valuable and testable on its own, with the acceptance criteria that would prove it. Say which split you would not recommend and why. Source: only the story below.*

The second one is not more clever. It is more specific about what you already know.

4. The one key most people never find

In Copilot, type  and start typing the name of a file, person, meeting or email. You can attach a single file or a whole folder. Inside a SharePoint site you can reference up to ten files or pages. Checked September 2026.

5. Where your prompts actually go

When you are signed in with your work account, prompts and responses are covered by enterprise data protection, and Microsoft states they are not used to train the foundation models. Access is scoped by your existing permissions, so Copilot cannot show you a document you could not already open yourself. For users in the European Union there are additional European Union Data Boundary safeguards.

The caveat that matters. All of that describes a work account signed in with your company identity. A personal account is a different product with different terms.

Ethical and data concerns are named by 35% of practitioners. This section is the answer to most of them. This is practical guidance, not legal advice, and your company's own rules sit on top of it.

6. It will lie to you in your own house style

It will state things that are not true, confidently and in your own house style. Three things need checking every single time:

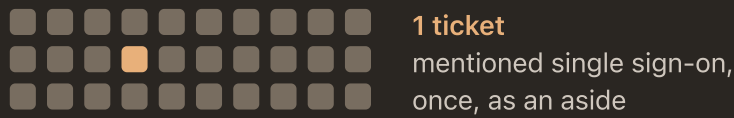
- **Numbers.** Check every figure you did not give it yourself.
- **Names.** People, products, teams, systems.
- **Anything stated as a fact about a user or a customer.** This is the dangerous one, because it is the hardest to spot and the most expensive to put in a backlog.

What this looks like in practice. You paste thirty support tickets and ask for the themes. Back comes: "Customers consistently request single sign-on." You go looking, and one ticket mentioned it once, as an aside. Nothing was invented exactly. A single signal was promoted to a pattern, in confident language, in a sentence that could become an epic. That is the failure mode to expect.

One mention is not a pattern

The failure mode to expect when you ask for themes

30 SUPPORT TICKETS PASTED IN



WHAT CAME BACK

"Customers consistently request single sign-on."

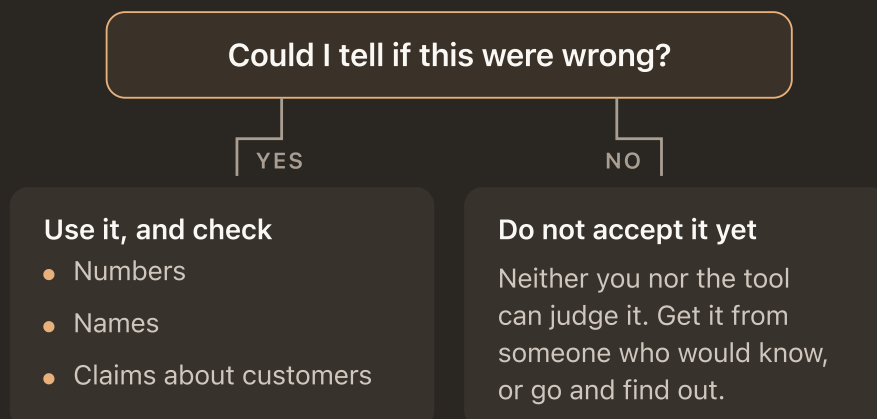
One ticket in thirty, reported as consistent.

Field note. The railure is rarely an invented fact. It is a real-looking number with no source behind it, stated with total confidence. We catch these because we check the source every time, not because the output looks doubtful. It never looks doubtful.

The decision aid. Before you use an output, ask one question: *could I tell if this were wrong?* If yes, use it and check it. If no, you are not the right person to accept that output yet, and neither is the tool.

Before you use the output

One question, asked every time



Part 1: four habits that separate method from luck

Part 0 was how the tool behaves. These four are how you behave, and they are the whole difference between an answer that worked once and a result you can repeat.

Rule 1. Give it your own material

The difference between a generic answer and a useful one is almost always what you put in, not how you phrase the question.

Paste the actual ticket, not your summary of it. The actual transcript, not your memory of the refinement session. The actual acceptance criteria you rejected, not a description of them.

Rule 2. Know what must never go in

Three questions you can answer in a second:

1. **Which account am I in?** Work identity, or personal. See Part 0, item 5.
2. **Would I put this in an email to a supplier?** If not, it does not go in a chat window either.
3. **Is it someone else's personal data?** User research recordings, support tickets with names in them, anything from a customer system. Anonymise or do not paste.

Rule 3. Check it before it leaves you

Check the same three things every time: numbers, names, and anything stated as a fact about a user. The list is in Part 0, item 6.

The moment that matters is **before it leaves you**, not before it ships. Once a plausible wrong sentence is inside a backlog item, it stops being an AI problem and becomes something a team builds.

Rule 4. Keep what works

When an answer comes out genuinely good, save the prompt that produced it, with the parts that change marked in square brackets, wherever your team already looks. Next time the same job takes a minute instead of twenty.

72% of practitioners report reduced cognitive load from using AI. Saved prompts are how you get that as a habit rather than a one-off.

Part 2: the five things that are actually yours

Rules are easy to agree with and easy to forget. Here they are against the five accountabilities the Scrum Guide names as yours, in the order the work happens, from writing an item to showing what it changed.

Chapter 1. Why your items come back from refinement

The need. An item goes into refinement and the session is spent filling in things you could have written down first.

Where it genuinely helps.

- Turning a rough note into a first item you then rewrite, which is faster than facing an empty field.
- Listing the questions a developer would ask before picking the item up, which is the fastest way to find what you left out.
- Rewriting an item that got bounced in refinement, using the objections as input.

What it cannot do. It cannot know why the item matters. That is the part you supply, and it is the part that makes the item worth building.

One prompt. Works on any tier, because you paste the content in.

```
Goal: list the questions a developer would ask before starting the
backlog item below.
Context: [product], [team], the item is meant to fit inside one
sprint.
Expectations: the questions only, ordered by how much they would
change the estimate, each tagged days, hours or unknown for how much
estimate the answer would move, so I can check the order. Do not
answer them and do not rewrite the item.
Source: only the item below.

[paste the item]
```

Field note. These tools save a serious amount of time on the parts of an item you should have settled before the session. They do not remove the questions, and they should not. Where the team works well and the goal is clear, good questions still come up about the aspect nobody considered, and that is exactly right. Writing a strong epic, feature or story is team work. The tool clears the avoidable questions so the session can reach the ones worth having.

Your first step. Run it on the item currently at the top of your backlog. Anything on that list you cannot answer is what refinement was going to spend twenty minutes finding.

Chapter 2. Refinement is not a group reading session

The need. Refinement becomes forty minutes of the team reading a story for the first time.

Where it genuinely helps.

- Proposing splits for an item that is too large, which you then accept or reject.
- Drafting acceptance criteria that are actually testable rather than restatements of the title.
- Finding the assumption buried in a story, the thing everyone reads past.

What it cannot do. It cannot tell you which split delivers value on its own. It will happily split by technical layer, which produces items nobody can ship. That judgment is the work.

One prompt.

Goal: propose three ways to split the story below into smaller items.
Context: [product]. Each resulting item must be independently valuable to a user and finishable inside one sprint.
Expectations: for each split, the resulting items in one line each, and what a user could do after the first one alone. Then name the split you would not recommend and say why.
Source: only the story below.

[paste the story]

Your first step. Take the largest item in your backlog and run it. The split you would not recommend is usually the one the team would have

proposed.

Chapter 3. Ordering is the part nobody can do for you

The need. What sits at the top is there because someone pushed hardest, and you are the one who has to defend it anyway.

The Scrum Guide is explicit here: you may delegate the work but remain accountable, and the product owner is one person, not a committee.

Where it genuinely helps.

- Arguing the other side of an ordering decision before a stakeholder does it for you.
- Making the trade-off explicit, so what you are giving up is written down rather than discovered later.
- Checking an order against the Product Goal, which the Scrum Guide defines as a future state of the product that the team can plan against, and naming the items that no longer serve it.

What it cannot do, and this is the important one. It cannot order your backlog. A model that has never met your users, does not know what your organisation can absorb and has not sat in the conversation is guessing, fluently. An AI draft never becomes the accountability. Use it to test an order you already hold, never to produce one.

Field note. What actually wins an argument about order is analytical data behind the position, and this is where AI earns its place: finding validated sources that support building a feature, or that show the assumption behind it was wrong and it should be dropped. Note which way round that runs. The value is in being willing to lose the argument to the data.

The specific danger. Generating backlog items in bulk. It is the easiest thing to do with these tools and the least useful. Volume is not value, and every item you add is something the team must read, estimate and eventually delete.

One prompt.

```
Goal: make the strongest case against the order below.
Context: the Product Goal is [goal]. These are the next [N] items in
the order I have put them in: [items].
Expectations: up to three of the strongest objections, each naming
what would have to be true for the objection to win. Then list any
item that does not serve the Product Goal at all, or say none if they
all do.
Source: only what I have written.
```

Your first step. Run it before your next stakeholder conversation about priorities, not after.

Chapter 4. Planning starts before the meeting does

The need. Planning opens with people picking tickets and nobody has said what the sprint is actually for.

The first topic of sprint planning is why this sprint is valuable, and the Scrum Guide puts proposing that value increase with you.

Where it genuinely helps.

- Turning a set of selected items into a candidate sprint goal in one sentence, which you then correct.
- Asking whether the selected items actually hold together, or whether they are just the next things in the list.

What it cannot do. It cannot decide what is valuable. A model that has never met your stakeholders is guessing. What it can do is make you articulate the goal before the meeting, so you arrive with a sentence instead of a list.

One prompt.

```
Goal: draft a candidate sprint goal from the items below.
Context: [product], a [length] sprint. The Product Goal is [goal].
Expectations: one sentence naming the outcome a user or the business
gets. It must not name any item, feature or component from the list.
If every item belongs to one feature and the outcome cannot be stated
without naming it, say so and name it rather than writing something
vague. Then say which items do not contribute to that goal, and
whether the set holds together as one objective or is really two.
Source: only the items below.

[paste the selected items]
```

Your first step. Draft it before planning and bring it as a proposal. A sentence to argue with beats a blank room.

Chapter 5. What got done, and what actually changed

The need. The room stays quiet, because nobody can tell what is different for a user who was not in it.

Where it genuinely helps.

- Turning what was built into what changed for a user, which is a different sentence and the one stakeholders need.
- Preparing for the questions you will be asked, including the uncomfortable one about what slipped.
- Listing what the sprint did not settle, which the standard recap tends to leave out.

What it cannot do. It cannot demonstrate the increment. The Scrum Guide is blunt about this: the sprint review "is a working session and the Scrum Team should avoid limiting it to a presentation." A polished narrative that hides a thin increment costs you more than it buys.

One prompt.

```
Goal: turn the completed items below into what changed for a user.
Context: [product], [stakeholders and what they care about]. The
Product Goal is [goal].
Expectations: one line per item saying what someone can now do that
they could not before. Mark any item where you cannot tell, rather
than guessing. Then list the three questions a sceptical stakeholder
would ask.
Source: only the items below, plus the stakeholders I named in
Context. This applies to the questions as well: do not reference any
story, ticket, discussion or prior release that is not listed here.

[paste the completed items]
```

Your first step. Run it on your last completed sprint. Any item where it cannot say what changed for a user is worth a conversation with the team.

Part 3: the prompt pack

Every prompt below uses the four parts from Part 0, item 3. Square brackets mark the parts you change. Each says what it needs, and "any tier" means it works even on Copilot Chat (Basic), because you supply the content yourself.

BACKLOG HYGIENE, ANY TIER

Find what should not be there.

Goal: review the backlog items below against the Product Goal.
Context: the Product Goal is [goal].
Expectations: four lists. Items that clearly serve the goal, items that do not, items whose fit is arguable with three words on why, and items that are duplicates or near-duplicates of each other. An item may appear in more than one list. Quote the item titles, do not paraphrase them.
Source: only the items below.

Find the items nobody can start.

Goal: list the backlog items below that have no testable acceptance criteria.
Expectations: the item titles only, and for each every category that is missing: scope, measurable outcome, or definition of done. Name the categories only. Do not give an example of the criterion.
Source: only the items below.

Find the buried assumption.

Goal: name the assumptions the story below depends on.
Context: [product], [who the user is].
Expectations: assumptions only, each with how we could check it cheaply. Do not propose solutions.
Source: only the story below.

TEAMS, NEEDS MICROSOFT 365 COPILOT

Copilot in Teams reaches back over the last 30 days of meetings, which is easy to miss.

Turn a refinement session into decisions.

Goal: extract what this session decided about each item.
Expectations: a table of item, decision, open question. Every item gets a row; leave either column blank where there is nothing to put in it. Then list the items with a blank decision again under a heading titled undecided. Do not invent decisions.
Source: this meeting only.

OUTLOOK, NEEDS MICROSOFT 365 COPILOT

Find the request buried in a long thread.

Goal: tell me what, if anything, this thread asks me to change in the backlog. If it asks for a commitment, a date or an estimate instead, say that and do not invent a backlog item.

Expectations: the request in one line, everyone asking for it, what they say the value is, and whether they have said anything about urgency.

Source: this thread only.

Part 4: staying current on thirty minutes a week

The training gap at the top of this guide is not going to be closed by waiting for a course. Here is a routine that costs nothing and needs nobody's approval.

Thirty minutes a week

Thirty minutes a week

No budget, no approval, no course

10 min	15 min	5 min
What changed. One source, rotating.	A real task from your week, done again with the tool. You already know what good looks like.	Write it down.

The middle block is the one that teaches. The other two make it stick.

Ten minutes, what changed. One source from the list below, rotating. Not all of them.

Fifteen minutes, on your own real work. Take something you did this week, a split, an item, a sprint goal, and do it again with the tool. Not an exercise, the actual work, where you already know what good looks like.

Five minutes, write it down. One line in your saved prompts: what worked, what did not. This is Rule 4.

Where to look, and how often

QUESTION	SOURCE	HOW OFTEN
What changed in Copilot	Microsoft 365 Copilot release notes on Microsoft Learn	Monthly
What changed in the models	The providers' own release notes	When something ships
What changed in the framework	The Scrum Guide at scrumguides.org , which is versioned and short	When a version lands
What changed in the rules, in the EU	The European Commission's AI Act pages. The AI literacy duty in Article 4 has applied since 2 February 2025	Quarterly

Do not treat vendor blogs, news aggregators or social threads as primary sources. Every number in this guide had to survive being traced back to the organisation that published it, and most of what circulates does not.

The weekly

If you would rather not do the first ten minutes yourself, we do it and send one email a week. One change that matters and what to do about it, one prompt of the week, one thing worth reading and why, and one line on what we got wrong the week before. Five minutes. Unsubscribe whenever.

Sign up at learnslice.com/ai-guides. If this was forwarded to you, that is also where the other guide is.

Sources

Every source below was checked on 17 September 2026 and resolves.

- **The Scrum Guide**, for every statement about what a product owner is accountable for and what happens in each Scrum event. Quoted, not interpreted. scrumguides.org/scrum-guide.html
- **AI4Agile Practitioners Report 2026**, 289 agile practitioners across more than 20 countries, published February 2026, for every

percentage about AI use among practitioners. Two limits, stated plainly. It is a small, self-selected sample with no breakdown by role, so read it as a direction of travel. And unlike every other source here, we have not read the report itself: its publisher's page would not open to us, so the figures are taken as reported consistently by two independent sources. scrum.org/resources/blog/ai4agile-practitioners-report-2026

- **Microsoft**, product documentation, for Copilot licence tiers, grounding, data protection, file referencing and prompt structure, and for the Jira and Confluence connectors: an administrator must deploy them and they enforce the source system's own permissions. learn.microsoft.com/microsoft-365/copilot/microsoft-365-copilot-overview and [/copilot/connectors/jira-cloud-overview](https://learn.microsoft.com/microsoft-365/copilot/connectors/jira-cloud-overview)
- **European Commission**, for the AI Act and the Article 4 AI literacy duty applicable since 2 February 2025. digital-strategy.ec.europa.eu/en/faqs/ai-literacy-questions-answers

No figure here is estimated, modelled or taken from a vendor's marketing material. Where the evidence is thin, this guide says so rather than filling the gap.



Alesia Kunz, CEO of LearnSlice

17+ years in software engineering as a product manager and product owner. This guide is built on two things: experience from real teams and projects, and published research. The field notes are the experience. Every figure is traced to the organisation that published it, and every source is listed above. The two are styled differently on purpose, so you can always tell which you are reading.

What we do

Nothing before this page was an advertisement. This page is, and it is the only one.

When the limit is the tool, not you. Every chapter above ends with something the tool cannot do, and most of those limits are the same limit: the model cannot read your own material. It cannot search your tickets, your documents or your decisions, so it answers from the public web and from whatever you paste. We build the layer that fixes that, grounded on a company's own content, hosted where the company needs it hosted. If that is the wall you keep hitting, it is worth a conversation.

Field note. The assumption we meet most often is that an AI tool will read a company's own documents and structure them correctly by itself. Two things have to be settled first. Whether that data may be shared with a public AI provider at all, most of which sit outside the EU, which is a GDPR question before it is a technical one. And what the data is for: the use case, the goal and the expected result are inputs, not outputs. The second assumption is that results will be right immediately. The more complex the use case, the more fine-tuning it needs, and that is time and effort rather than a switch.

AI Mentor, for a team rather than a person. The routine and the activation plan in this guide work for one person with thirty minutes a week. Running the same thing across a team, with content matched to your sector and a record of who actually learned what, is what AI Mentor does. It is the same discipline as this guide, delivered as a product.

If you picked one of those two on the download form, we have your answer and will come back to you on it. If you picked neither and changed your mind, or this was forwarded to you, learnslice.com/ai-guides is where to say so.